

benchmark_v2 有效性审查与复跑报告

日期：2026-05-05 目标：说明 benchmark 当前到底测到了什么、没测到什么，并给出一轮新的本地结果

1 报告定位

这份报告不是系统总验收，也不是单纯展示一个通过率。

它分四层叙事：

- 结构层：fixture / runner / reporting 是否可用
- 轻量回归层：不依赖本地 LLM 写入链时，benchmark 能否稳定暴露实时分类和干扰误判
- 深度子集层：开启 FULL_WRITE + DEEP_EVAL 后，写入侧卡片检查是否能打出真实失败
- 有效性层：按公开 memory benchmark 常见原则，判断这些结果可以支持哪些结论，不能支持哪些结论

2 审查依据

本次有效性审查参考了近年的几类 memory benchmark 设计原则：

- LongMemEval：把长期记忆拆成 extraction、multi-session reasoning、temporal reasoning、updates、abstention 等能力，而不是只看单一命中率
- EverMemBench：强调 multi-party、multi-group、长时间跨度、跨主题交错和角色差异
- MemBench：强调不要只看 accuracy，还要看效率、容量和使用场景差异
- StructMemEval：提醒不能只测“记没记住”，还要看记忆结构是否组织得合理
- MemoryAgentBench：强调 incremental multi-turn interaction 和 selective forgetting / update

因此，本报告重点检查四件事：

- 是否存在负样本、近似误判样本、跨群漂移样本
- 是否区分轻量路由能力与深度写入能力
- 是否把 long-term recall、update、evidence、relation 分层看待
- 是否避免把 filtered run 或轻量代理指标误写成总能力结论

3 本轮运行内容

层次	作用	本轮输出
结构校验	检查 source/runtime schema 是否一致	validate_fixture 通过
单元测试	检查 runner、reporting、replay adapter 兼容性	11/11 通过
轻量全量回放	主看实时分类、忽略规则、困难样本区分能力	生成 light_full_results.json
深度子集回放	主看 FULL_WRITE 后的卡片命中与环境退化信号	生成 deep_oc_launch_ops_results.json

```
本轮实际执行命令:conda run -n feishu-ai-p0 python benchmark_v2/validate_fixture.py
conda run -n feishu-ai-p0 python -m unittest
tests.test_benchmark_v2_runner tests.test_benchmark_replay
conda run -n feishu-ai-p0 python -m benchmark_v2.dual_channel_runner
benchmark_v2/full_demo_case_v2.json
BENCHMARK_V2_DEEP_EVAL=1 FULL_WRITE=1
conda run -n feishu-ai-p0 python -m benchmark_v2.dual_channel_runner
benchmark_v2/full_demo_case_v2.json --chat oc_launch_ops
```

4 当前样本池分类

当前 runtime fixture 共 24 个 batch。下面的分类是**标签覆盖计数**，不是互斥分桶，所以相加会超过 24。

类别	覆盖 batch	说明
结构 / 分块样本	2	progress、sparse window / lookback
干扰 / 路由样本	15	noise、query、schedule、task、parallel、topic_boundary、near_miss 等
冲突 / 更新样本	4	refine、supersede、conflict、long-term update
长线追问样本	7	source、version、summary、topic_list、long-term recall
非工程协作样本	5	发布运营、客服口径、会务组织、答辩物料

本轮新增后的困难样本特征：

- 带时间词但本质是发布策略或风险口径，而不是 schedule
- 非 @ 机器人的人对人复述请求，防止 query 过敏
- 直播备用链接预案，防止误判成日程
- 显式排除后续改口的 long-term recall
- 同一时间窗口内碎片化多线程收口，防止 topic merge 过度

5 当前会生成哪些指标

5.1 结构层

指标	意义
validate_fixture 是否通过	source/runtime schema、source-only 字段剥离、基本字段完整性
单元测试通过数	runner / reporting / replay adapter 的局部可用性

5.2 轻量全量层

指标	意义
passed_batches / total_batches	当前样本包上的总体通过率
interference_metrics.batch_pass_rate	干扰相关 batch 总体通过率
realtime_action_match_rate	实时动作是否符合 fixture 标注
write_count_match_rate	写入侧产出数量是否符合轻量预期
ignore_rule_match_rate	被标记为应忽略的消息是否确实被隔离
difficult / multi_intent / near_miss batch pass rate	困难样本、混合意图样本、近似误判样本的区分能力
realtime_action_distribution	当前 fixture 下动作分布是否失衡
performance.*	回放延迟、写入延迟、吞吐

5.3 深度子集层

指标	意义
expected_memory_cards[*]	写入后卡片内容是否命中预期关键词
expected_relations[*]	关系抽取是否命中预期类型和前后卡片
expected_evidence_checks[*]	来源 / 证据是否能回到预期 message ids
avg / p95 write latency	FULL_WRITE 真实耗时
Graphiti / Stage 2 退化日志	区分环境降级和系统能力失败

6 有效性审查

6.1 1. Construct Validity: 当前 benchmark 是否在测对的东西

结论：基本成立，但只对一部分能力成立。

成立的部分：

- 当前 fixture 已经能测到实时分类的现实误判，不再只是功能 smoke test
- 已经有 near-miss、cross-group、long-gap recall、non-engineering 场景
- 轻量层和深度层的能力口径已经分开

仍然不足的部分：

- conflict / relation 目前在轻量全量模式下只统计 batch 覆盖，不统计真实 relation 命中
- case 级 final recall 这轮没有跑完整 deep，因此不能宣称终态 long-term retrieval 已验证
- 当前 fixture 仍是单语言、单主项目家族，外部有效性有限

6.2 2. Discriminative Validity: 当前 benchmark 能不能把强弱区分开

结论：可以，但目前主要体现在 realtime routing，而不是深度 relation。

证据是这轮轻量全量结果里：

- 总体通过率不是 100%，而是 21/24
- 干扰相关 batch 通过率是 13/16 = 0.8125
- multi-intent 样本通过率是 5/7 = 0.7143
- near-miss 样本通过率是 3/6 = 0.5

这说明样本已经能把“普通规则命中”和“现实近似误判”拉开。

6.3 3. Internal Validity: 结果是否会被口径混淆

结论：比之前清楚了，但仍需在报告里明确限制条件。

需要明确写清楚的点：

- 标签分类是重叠计数，不是互斥分桶
- filtered deep run 会跳过 case-level eval，不能拿来证明完整终态 recall
- deep_eval_disabled 时，relation 命中率应视为 skipped，而不是 0
- Graphiti 未初始化会让 Stage 2 降级到 Jaccard，这种结果不能和完整环境混读

6.4 4. External Validity: 离真实生产还有多远

结论：离“生产级评测体系”更近了，但还不是完整生产外推。

当前优点：

- 已包含多群、多角色、非工程协作、跨群漂移和长线追问
- 已显式引入本体构造、时间基底、线程槽位和 iceberg 隐层

当前不足：

- 样本规模仍小，只有 24 个 batch
- 中文单域为主，缺少多语言和多租户
- FULL_WRITE 深度评测还没有稳定跑完整 case
- relation / evidence / topic summary 的深度成功率还没有形成完整统计面

7 复跑结果

7.1 A. 结构层结果

检查项	结果	说明
fixture 校验	通过	source/runtime schema 一致，source-only 字段剥离正常
单元测试	11/11	runner、reporting、replay adapter 基本可用

7.2 B. 轻量全量回放结果

指标	结果	分母	解释
总体 batch 通过率	87.5%	21/24	不再是全绿，说明样本已有区分度
failed checks	3	70 项检查	失败集中在 realtime routing
干扰相关 batch 通过率	81.25%	13/16	干扰样本已能稳定暴露问题
realtime action 匹配率	81.25%	13/16	失败点就是当前规则分类缺口
write count 匹配率	100%	16/16	轻量写入数量口径基本稳定
ignore rule 命中率	100%	5/5	负样本隔离目前稳定
difficult batch 通过率	100%	6/6	当前“难”的定义仍偏结构难，不完全等于语义难
multi-intent batch 通过率	71.43%	5/7	混合意图仍明显拉低成功率
near-miss batch 通过率	50.0%	3/6	当前最有价值的区分信号

性能口径：

- case runtime: 6.559 ms
- avg realtime latency: 0.018 ms
- avg write latency: 0.080 ms
- p95 write latency: 0.137 ms

这层能得出的结论：

- benchmark 已经能稳定暴露实时分类误判
- 误判集中在“带时间词的发布/风险口径”与“风险预案”
- 这层不能证明 relation、evidence、终态 recall 成功

7.3 轻量层失败 batch

batch	失败含义
batch_016_product_launch_ops	“晚上八点灰度开放给老师试用”被规则判成 schedule, 但 benchmark 认为它应属于发布策略口径
batch_017_meeting_org_external	“直播挂了就切备用腾讯会议链接”被规则判成 schedule, 但 benchmark 认为它是风险预案 / 会务规则
batch_022_release_policy_time_boundary	同一批中“晚上八点灰度开放”与“今晚七点前补完 FAQ”被判成 schedule + schedule, 而 benchmark 期望 policy + task

7.4 C. 深度子集回放结果 (oc_launch_ops)

指标	结果	分母	解释
batch 通过率	0%	0/3	三个发布运营子样本全部暴露问题
failed checks	5	15 项检查	失败不只在 routing, 也在卡片命中
realtime action 通过率	33.3%	1/3	深度模式下仍有 realtime routing 问题
write result count 通过率	100%	3/3	FULL_WRITE 产物数量还在, 但内容不稳
memory card 命中率	57.1%	4/7	卡片内容命中明显不稳
ignore rule 命中率	100%	1/1	负样本隔离仍成立

性能口径:

- case runtime: 37110.662 ms
- avg realtime latency: 0.030 ms
- avg write latency: 12368.586 ms
- p95 write latency: 15970.542 ms

环境与退化信号:

- 日志出现 Stage 2 unavailable, falling back to Jaccard
- 日志出现 Graphiti not initialized
- 本次是 filtered run, 所以 case-level deep eval 被跳过

7.5 深度层失败解释

batch	失败含义
batch_016_product_launch_ops	除了 realtime 误判外, 公告措辞卡片没有稳定命中“辅助初筛 / 不要写完全替代人工”
batch_018_customer_policy_boundary	客服口径卡片没有同时覆盖“仅供参考 / 人工复核 / 不要承诺 / 稍后重试”
batch_022_release_policy_time_boundary	realtime 误判仍在, 同时“辅助初筛”口径卡片未稳定命中

这层能得出的结论:

- benchmark 已经能把写入侧内容问题打出来, 不只是测 routing
- 当前 FULL_WRITE 下, 发布运营类场景的记忆卡片稳定性不足
- 但这层仍不能证明 Graphiti / Stage 2 / 完整 case-level long-term recall 的最终水平

8 最终判断

可以肯定的:

- 当前 benchmark_v2 已经具备结构化、分层次、可复跑、能暴露真实问题的基本条件

- 当前样本不再只是“能跑通”，而是已经能有效区分普通样本和 near-miss / multi-intent 样本
- 对你们当前阶段最有价值的是：它已经足够拿来压 realtime 分类与写入卡片质量

不能过度声称的：

- 不能说“冲突更新已经验证成功”，因为 relation 命中率这轮是 skipped
- 不能说“终态 long-term recall 已完整验证”，因为没有完整 case-level deep eval
- 不能说“系统已生产级可用”，因为 FULL_WRITE 在非工程场景子集上仍明显失败，且 Graphiti 仍未初始化

下一步最该做的：

- 先用这 3 个轻量失败 batch 回归修 realtime 规则：发布窗口、风险预案、FAQ 截止的边界
- 再用 oc_launch_ops 子集压写入卡片质量，先把 7 个 memory card 检查中的 4/7 提上来
- Graphiti / Stage 2 初始化稳定后，再补一轮完整 deep case，届时再谈 relation / evidence / final recall 总结论